KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

Deemed to be University U/S 3 of UGC Act, 1956

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024

A PROJECT REPORT
On

"LUMINA"

AI-Powered Data Analytics Platform

Submitted to
KIIT Deemed to be University
In Partial Fulfilment of the Requirement for the Award of

BACHELOR'S DEGREE IN
INFORMATION TECHNOLOGY

BY

Kshitij Singh	**2306289**
Rajiv Chandak	**2306302**
Syed Aarish Aabdi	**2306359**
Mohammad Farooque Ahmed	**2306033**

UNDER THE GUIDANCE OF
Dr. Nibedan Panda

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA -751024

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
Deemed to be University U/S 3 of UGC Act, 1956

CERTIFICATE

This is to certify that the project entitled

"LUMINA"

AI-Powered Data Analytics Platform

submitted by

Kshitij Singh	**2306289**
Rajiv Chandak	**2306302**
Syed Aarish Aabdi	**2306359**
Mohammad Farooque Ahmed	**2306033**

is a record of Bonafide work carried out by him, in the partial fulfillment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science &Engineering OR Information Technology) at KIIT Deemed to be university, Bhubaneswar. This work is done during the year 2025-2026, under our guidance.

Date: 10/4 /26

Dr. Nibedan Panda
Project Guide

Acknowledgements

I am profoundly grateful to Dr. Nibedan Panda of the School of Computer Engineering, KIIT University for his expert guidance and continuous encouragement throughout to see that this project has rights its target since its commencement to its completion.

Signature of GROUP MEMBERS

_____

ABSTRACT

Generating insights from raw data is a critical factor for modern business growth. However, business users often lack the technical skills to write SQL or Python, giving insights behind manual, repetitive spreadsheet analysis. This mini-project presents **Lumina**, an intelligent, AI-powered data analytics platform that allows users to interact with Excel and CSV datasets using natural language.

The system utilizes Google Gemini 2.0 Flash alongside a robust Python and pandas backend to dynamically generate and execute code based on user queries. To ensure absolute data security and operational stability, Lumina features a Dual-Layer Security Sandbox utilizing Abstract Syntax Tree (AST) analysis to block malicious operations, paired with a self-healing retry loop that automatically corrects code execution errors.

Furthermore, the platform features a zero-configuration Insight Engine that instantly generates key performance indicators, correlation heatmaps, and data distributions upon file upload. An integrated cross-session SQLite database powers an Entity Relationship Graph, allowing the system to map correlations and shared keys across multiple datasets over time. Deployed via Streamlit and visualized with interactive Plotly charts, Lumina successfully bridges the gap between complex data science and accessible, no-code business intelligence.

Keywords: Data Analytics, Large Language Models (LLM), Generative AI, Google Gemini, Python, Streamlit, Abstract Syntax Tree (AST), Natural Language Processing.

Table of Content

CHAPTER 1

INTRODUCTION

1.1 The Problem Statement In the modern business ecosystem, data is abundant, but actionable insights remain locked behind technical barriers. Business users, managers, and domain experts often lack programming skills (such as SQL or Python) required to query their own data effectively. Consequently, insights are gated behind technical teams, leading to slow turnaround times. Furthermore, manual analysis using traditional spreadsheet software is repetitive, highly prone to human error, and requires specialized formula knowledge. Existing solutions also suffer from fragmented tooling, where users must switch between separate applications for natural language Q&A, visual analytics, and relationship discovery.

1.2 Proposed Solution: What Lumina Does To address these challenges, this project introduces **Lumina**, an AI-Powered Data Analytics Platform. Lumina democratizes data analysis by allowing users to interact with Excel and CSV files using plain English. The platform acts as a bridge between raw data and insights. Upon file upload, Lumina automatically profiles the data, generating key performance indicators (KPIs), distributions, and correlation heatmaps. Users can then ask conversational questions (e.g., "Show me the monthly sales trend by region"), and the system uses Google Gemini 2.0 Flash to autonomously write, securely execute, and render the corresponding Python code to produce interactive charts and tables.

1.3 Scope and Objectives The primary objective of this project is to develop a robust, secure, and user-friendly web application for automated data analysis. The key scopes include:

- Developing a Natural Language Interface for data querying.
- Implementing a secure, sandboxed execution environment to run AI-generated Python code safely.
- Creating an automated Insight Engine that requires zero manual configuration.
- Building a cross-session Entity Relationship Graph to map correlations across different datasets over time.

CHAPTER 2

BASIC CONCEPTS / LITERATURE REVIEW

2.1 Natural Language Processing in Data Analytics Natural Language Processing (NLP) has significantly transformed how humans interact with machines. In the context of data analytics, NLP allows for the translation of human language into executable database queries or scripts. This transition shifts the paradigm from "writing code to get answers" to "asking questions to get answers," dramatically lowering the barrier to entry for business intelligence tasks.

2.2 Large Language Models (Google Gemini 2.0 Flash) Large Language Models (LLMs) are the backbone of modern generative AI applications. For this project, Google Gemini 2.0 Flash was selected. The "Flash" variant of the Gemini architecture is optimized for high-speed, low-latency reasoning. This is crucial for Lumina, as the system relies on rapid code generation and multi-step self-healing retry loops that must execute seamlessly without degrading the user experience through long loading times.

2.3 Secure Code Execution & AST Sandboxing A major challenge in utilizing LLMs to generate executable code is the inherent security risk; the AI might hallucinate malicious or destructive commands. Abstract Syntax Tree (AST) analysis is a computer science concept used to parse code into a structural tree before execution. By evaluating the AST, systems can statically detect and block unauthorized imports (like `os` or `sys`) or built-in functions (like `exec` or `eval`), ensuring that untrusted AI-generated code is executed safely within a strictly confined environment.

2.4 Data Visualization and Graph Theory Effective data visualization is critical for interpreting analytical results. Plotly is utilized in this project for its ability to render highly interactive, web-native charts. Furthermore, Graph Theory is applied to map relationships between data columns. By calculating statistical metrics like Pearson Correlation for numeric data and Cramér's V for categorical data, the system constructs an Entity Relationship Graph, representing data columns as nodes and their correlations as edges.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 Primary Workflow & Architecture Diagram Lumina is built on a stateless architecture where session state is managed via the Streamlit frontend. The primary workflow follows a distinct pipeline: The user inputs a query via the browser, which is routed through the Streamlit UI. The prompt is sent to the Gemini API, which generates a Python/pandas script. This script is passed through an AST-checked Sandboxed Executor. Finally, the executed output (a chart or table) is rendered back to the user.

**

3.2 The data ingestion pipeline is designed for seamless processing. When an `.xlsx` or `.csv` file is uploaded, the Excel Processor module extracts the schema and determines data types. This structured data is then fed into the Insight and Graph Engines, which utilize `pandas`, `scipy` statistics, and `networkx` to compute metrics.

3.3 Zero-Config Insight Engine A core design constraint was minimizing user friction. The Insight Engine operates with zero user configuration. The moment data is ingested; it calculates a 5-metric KPI strip (rows, columns, numeric columns, categorical columns, missing cells). It also automatically computes a Pearson correlation matrix rendered as a heatmap, alongside histogram grids for numeric distributions and bar charts for categorical frequencies.

3.4 Cross-Session Entity Relationship Graph Unlike standard analytical tools that treat every file upload in isolation, Lumina employs an SQLite database to persist in cross-session graph memory. Data columns are classified as specific node types (Numeric, Date Time, Text). Edges are formed based on Pearson $r > 0.3$, Cramér's $V > 0.1$, or shared column names, allowing a cumulative knowledge graph to build over time.

CHAPTER 4

IMPLEMENTATION

4.1 Tech Stack The implementation relies on a modern, Python-centric technology stack:

- **Backend:** Python 3.11 serves as the core runtime.
- **AI Integration:** Google Generative AI SDK (Gemini 2.0 Flash) is used for natural language understanding and code generation.
- **Data Processing:** `pandas 2.1` and `numpy 1.26` handle dataframe manipulations, while `scipy` is used for statistical correlations. SQLite handles graph persistence.
- **Frontend:** Streamlit 1.35 provides the web UI framework, augmented with custom CSS for a dark-mode design system. Plotly 5.22 handles interactive charting.

4.2 Natural Language Q&A Pipeline The chat interface operates as the primary user interaction point. The pipeline takes the user's text question and wraps it in a system prompt containing the dataset's schema (column names and data types). Gemini generates a Python script utilizing a pre-injected namespace containing safe libraries (e.g., `pd`, `px`, `go`).

4.3 Implementing the Self-Healing Code Loop Because AI-generated code can occasionally fail due to hallucinated syntax or incorrect data assumptions, Lumina implements a self-healing retry mechanism. If the Sandboxed Executor catches an exception during runtime, the exact Python traceback is captured. This error message is automatically fed back to Gemini with instructions to correct the code. This loop runs up to two times silently, drastically improving the success rate of queries without user intervention.

4.4 Implementing the Dual-Layer Security Sandbox Security is implemented in two distinct layers to prevent data mutation or system compromise:

1. **Layer 1 (Static AST Analysis):** All code is parsed using `ast.parse()`. If malformed code or blocked imports (`os`, `subprocess`, `urllib`) are detected, a SecurityError is raised instantly.
2. **Layer 2 (Restricted Runtime):** The execution environment utilizes an explicit 40-item allowlist of safe built-in functions (e.g., `len`, `sum`, `max`). The original dataset is protected by ensuring the AI only interacts with a shallow copy (`df.copy()`).

4.5 Testing & Verification To ensure enterprise-grade stability, the system was built using Test-Driven Development (TDD) principles. A comprehensive suite of 98 Pytest tests was implemented across all 6 core modules, covering everything from AST isolation logic to missing value handling in the Insight Engine.

Test Case ID	Module / Component	Test Scenario (Input/Action)	Expected Result	Status
TC-01	Data Pipeline	Upload a valid `.xlsx` file containing multi-sheet data.	Schema successfully extracted, data types coerced, and loaded into pandas dataframe.	Pass
TC-02	Insight Engine	System initiates zero-config profiling upon file drop.	KPI strip, Pearson correlation heatmap, and distribution histograms generate automatically without user input.	Pass
TC-03	Insight Engine	Upload a dataset containing null/missing values.	KPI strip accurately counts "Missing Cells"; graphs auto-adjust to exclude or impute nulls without crashing.	Pass
TC-04	Security Sandbox (AST)	AI generates code attempting to `import os` or `subprocess`.	Static AST analysis intercepts the blocked import and raises a `SecurityError` before execution.	Pass
TC-05	Security Sandbox (Runtime)	AI generates code that attempts to drop a column from the source data.	Code executes strictly on `df.copy()`; the original user dataset remains completely unmutated.	Pass
TC-06	Self-Healing Execution	Executed Python code throws a `KeyError` due to a hallucinated column name.	Traceback is caught and fed back to Gemini; AI corrects the code and successfully renders the chart within 2 retries.	Pass
TC-07	Natural Language Q&A	User prompts: *"Show me the sales trend by region."*	Gemini generates safe Plotly code; an interactive bar/line chart is rendered in the chat bubble.	Pass
TC-08	Graph Engine	Calculate correlations on a newly uploaded dataset.	System calculates Pearson $r > 0.3$ and Cramér's $V > 0.1$ to successfully form graph edges.	Pass

TC-09	Graph Engine (Cross-Session)	User refreshes page and uploads a second dataset with shared keys.	SQLite database persists previous nodes; new edges are drawn connecting the old and new datasets.	Pass
TC-10	AI Predictions	User triggers a prediction on the 'Revenue' column.	System outputs a forecast card containing a trend outlook, ranked causal evidence, and a confidence rating.	Pass

CHAPTER 5

RESULT ANALYSIS & UI WALKTHROUGH

5.1 Chat Interface Highlights The primary user interface features a sleek, dark-mode chat environment. When a query is successfully executed, interactive Plotly charts are rendered directly within the AI response bubbles. Below each response, the system generates context-aware follow-up suggestions (e.g., "Suggest: Compare by region?"), guiding users toward deeper analysis.

5.2 Auto-Generated Data Profile The Insights Page successfully renders a complete snapshot of the dataset health instantly upon upload. The Pearson correlation heatmap utilizes an RdBu_r color scale, allowing users to instantly identify strongly correlated feature pairs via hover tooltips. The system also dynamically handles wide datasets by making histogram grids scrollable.

5.3 Graph Page & Persistence The Graph Page visually validates the cross-session SQLite implementation. Users can view a Plotly network graph where nodes represent columns, color-coded by data type. Hovering over nodes reveals column statistics, and the edges visibly denote the strength of the relationship (e.g., r=0.85).

5.4 AI-Powered Predictions & Causality The Predictions Page demonstrates the platform's advanced reasoning capabilities. By leveraging Gemini in a purely analytical mode, the system outputs structured trend forecast cards. For each prediction (e.g., "Revenue will continue growing in Q2"), the system provides a confidence rating (High/Medium/Low) and a detailed causal analysis citing historical data evidence.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion The Lumina platform successfully fulfills its primary objective: eliminating the technical barrier to data analysis. By combining the natural language processing capabilities of Google Gemini 2.0 Flash with a robust, sandboxed Python backend, the system allows non-technical users to query, visualize, and understand their data instantly. The implementation of self-healing code execution and a dual-layer AST security sandbox ensures the platform is not merely a prototype, but a resilient, production-ready tool. Furthermore, the cross-session Entity Relationship Graph proves that intelligent systems can build cumulative intelligence over time, fundamentally improving how businesses interact with fragmented spreadsheet data.

6.2 Future Scope While the current iteration of Lumina is highly capable, several key enhancements are planned for the platform's evolution:

- **Streaming LLM Responses:** To improve user experience, the system will be upgraded to support streaming tokens, eliminating the loading spinner freeze during complex code generation.
- **Database Integration:** Moving beyond static `.xlsx` and `.csv` uploads, mid-term goals include direct integrations with PostgreSQL and MySQL databases for real-time querying.
- **Multi-User Environments:** Implementing robust authentication (OAuth) to provide users with isolated session sandboxes, ensuring data privacy in enterprise deployments.
- **Open-Source Model Transition:** To reduce API dependency and costs, future versions will explore replacing Gemini with a fine-tuned, locally hosted open-source LLM specifically trained on pandas code generation.

References

1. Zhao, X., Wang, Y., and Li, J., 2024. Large language models as data analysts: A comprehensive survey on natural language to data processing. *arXiv preprint arXiv:2402.11223*.

2. Google DeepMind, 2025. *Gemini 2.0 Technical Report: High-Speed Multimodal Reasoning and Code Generation*. Google AI Research.
3. McKinney, W., 2022. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
4. Chen, Y., Liu, H., and Zhang, M., 2025. Secure sandboxing of LLM-generated Python code via Abstract Syntax Tree (AST) analysis. *IEEE Transactions on Software Engineering*, 51(3), pp.412-428.
5. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. and Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning and self-correction. *Advances in Neural Information Processing Systems*, 36.
6. Richards, T., 2024. Rapid development of interactive data science applications utilizing Streamlit. *Journal of Data Science Engineering*, 9(2), pp.115-130.
7. Sievert, C., 2020. *Interactive Web-Based Data Visualization with R, plotly, and shiny*. CRC Press.
8. Wang, L., Chen, D., and Gupta, A., 2025. Zero-shot automated exploratory data analysis using generative AI. *ACM Transactions on Interactive Intelligent Systems*, 15(1), pp.22-45.
9. Dong, Y., Sun, Z., and He, K., 2024. Bridging the gap: Natural language to Python data analysis frameworks. *International Conference on Machine Learning (ICML)*, pp.3012-3025.
10. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V. and Zhou, D., 2023. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, pp.24824-24837.
11. Agresti, A., 2018. *An Introduction to Categorical Data Analysis* (3rd ed.). John Wiley & Sons. *(Reference for Cramér's V statistical methodology)*.
12. Owens, M. and Allen, G., 2023. *SQLite Database System Design and Implementation*. Apress.
13. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A. and Fung, P.C.F., 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), pp.1-38.

GROUP CONTRIBUTION REPORT

LUMINA: AI-POWERED DATA ANALYTICS PLATFORM

Kshitij Singh (2306289) | Rajiv Chandak (2306302) | Syed Aarish Aabdi (2306359) | Mohammad Farooque Ahmed (2306033)

Abstract:

This project focuses on developing Lumina, an intelligent data analytics platform that empowers users to interact with Excel and CSV datasets using natural language. The system utilizes Google Gemini 2.0 Flash for automated Python code generation, paired with a secure, sandboxed execution environment. It features a zero-configuration insight engine for instant data profiling, an SQLite-backed cross-session entity relationship graph, and interactive Plotly visualizations, effectively bridging the gap between raw data and actionable business intelligence.

1. Rajiv Chandak (2306302) - Data Pipeline & Graph Engine

Individual Contribution and Findings:

My primary focus was the integration of the Large Language Model (LLM) and the core backend orchestration. I was responsible for connecting the Google Generative AI SDK (Gemini 2.0 Flash) to our Python environment. I engineered the natural language Q&A pipeline, ensuring user prompts were accurately packaged with dataset schemas before being sent to the AI. My major architectural contribution was designing the "Self-Healing Code Loop." During implementation, I observed that the AI occasionally hallucinated invalid pandas syntax. By catching these runtime errors and feeding the traceback back to the AI for a maximum of two retries, the system's success rate in generating accurate code improved drastically.

Contribution to Project Report Preparation:

I was responsible for drafting the Introduction, System Architecture, and Future Scope chapters of the report. I also designed the primary workflow block diagrams that illustrate the data flow from the user to the LLM and back.

Contribution for Project Presentation and Demonstration:

For the presentation, I prepared slides related to the Problem Statement and System Architecture. During the demonstration, I am responsible for explaining the overall system flow and showcasing the AI's self-healing capabilities in real-time.

2. Syed Aarish Aabdi (2306359) - Frontend Architecture & Visualization

Individual Contribution and Findings:

My primary responsibility was the user interface and data visualization layers. I built the frontend using Streamlit, developing a responsive, stateless, dark-mode web application consisting of four distinct pages (Chat, Insights, Graph, Predictions). I integrated Plotly to ensure all AI-generated charts were highly interactive rather than static images. During development, I found that managing Streamlit's session state was critical to prevent the UI from resetting during LLM API calls. I also designed the chat interface, including the generation of dynamic follow-up question suggestions to guide the user's analytical journey.

Contribution to Project Report Preparation:

I authored the Result Analysis and UI Walkthrough sections of the project report. I was also responsible for capturing, formatting, and inserting all system screenshots to visually document the project's interface.

Contribution for Project Presentation and Demonstration:

I designed the overall slide deck's visual theme. During the demonstration, I will handle the live UI walkthrough, showing the panel how a user interacts with the Chat and Insights pages, and demonstrating the interactive nature of the Plotly visualizations.

3. Kshitij Singh (2306289) - AI Integration & Core Backend

Individual Contribution and Findings:

My role centered on data ingestion and automated statistical profiling. I built the Excel Processor module to handle file uploads, extract schemas, and coerce data types. I developed the Zero-Config Insight Engine using `pandas` and `scipy`, which automatically computes KPI strips, correlation heatmaps (using Pearson's r), and distribution histograms upon file upload. Furthermore, I engineered the Cross-Session Entity Relationship Graph. I implemented an SQLite database to store column relationships and utilized Cramér's V to measure categorical co-occurrences. I found that

persisting in this data across sessions allowed the system to build a highly valuable, cumulative knowledge graph of the user's business data.

Contribution to Project Report Preparation:

I wrote the Literature Review and the technical sections detailing the algorithms (Pearson correlation, Cramér's V) and database schema. I also structured the project's requirement specifications.

Contribution for Project Presentation and Demonstration:

I prepared the slides focusing on the Auto-Generated Data Profile and Graph Theory. During the demo, I will present the Insights Page and explain how the system maps node and edge relationships dynamically on the Graph Page.

4. Mohammad Farooque Ahmed (2306033) - Security Sandbox & Testing

Individual Contribution and Findings:

My core focus was on system security, reliability, and testing. Executing AI-generated Python code presents severe security risks. To mitigate this, I developed the Dual-Layer Security Sandbox. I implemented Static AST (Abstract Syntax Tree) Analysis to parse code before runtime, actively blocking malicious imports like `os` and `sys`. I also built a restricted runtime allowlist for safe built-in functions. To ensure system stability, I implemented a comprehensive Test-Driven Development (TDD) approach, writing 98 Pytest tests that provided full coverage across all 6 core modules. I observed that strict AST parsing prevented 100% of simulated malicious injection attempts during our security testing phase.

Contribution for Project Presentation and Demonstration:

I built the slides explaining the AST Layer and the restricted runtime environment. During the presentation Q&A, I will act as the technical lead for questions regarding system security, sandbox constraints, and the testing methodologies used to validate the platform.

Full Signature of Supervisor: _____

Full Signatures of Students:

_____ (Kshitij Singh)

_____ (Syed Aarish Aabdi)

_____ (Rajiv Chandak)

_____ (Mohammad Farooque Ahmed)